

SISTEMA ERP DE COMPRA E VENDA

Resumo técnico e guia de uso

Java Swing · JDBC · PostgreSQL/Supabase · MVC · DAO

Disciplina: Práticas de Programação Orientada a Objetos — UNIP

Projeto: MarketPlace-System-Java-SQL

Sumário

1	Objetivo e visão geral
2	Funcionalidades do sistema
3	Como configurar e usar
4	Arquitetura MVC
5	Padrão DAO — foco principal
6	Transações e consistência
7	ConnectionFactory, JDBC e Supabase
8	Banco de dados e relacionamentos
9	Segurança, permissões e recursos auxiliares
10	Roteiro de apresentação e conclusão

Ênfase do documento

A seção 5 detalha o padrão DAO, a interface GenericDAO, PreparedStatement, mapeamento de ResultSet, tratamento de erros e as transações usadas em Cliente, Compra e Venda.

1. Objetivo e visão geral

O projeto implementa um ERP desktop de compra e venda. Ele reúne cadastros, movimentação de estoque, compras, vendas, formas de pagamento, controle de usuários, históricos e relatórios. A interface é construída com Java Swing e os dados são persistidos em PostgreSQL hospedado no Supabase por meio de JDBC.

A proposta acadêmica demonstra orientação a objetos, banco de dados relacional e separação de responsabilidades. Os dois padrões estruturais mais importantes são:

MVC: separa interface gráfica, regras de negócio e dados.

DAO: concentra o SQL e o acesso JDBC em classes específicas, evitando que as telas conheçam detalhes do banco.

Resultado

Controllers e Views trabalham com objetos Java. A persistência fica isolada nos DAOs e a criação de conexões fica centralizada na ConnectionFactory.

Tecnologias utilizadas

Tecnologia	Uso no projeto
Java 17	Linguagem e recursos de orientação a objetos.
Java Swing	Telas, formulários, tabelas, menus e diálogos.
FlatLaf	Tema moderno claro/escuro para a interface Swing.
Maven	Dependências, compilação e geração do JAR executável.
JDBC	Comunicação entre Java e PostgreSQL.
PostgreSQL / Supabase	Banco relacional remoto e persistência dos dados.

2. Funcionalidades do sistema

Módulo	Funcionalidades
Autenticação	Login, sessão do usuário, perfil comum e administrador.
Cadastros	Usuário, cliente, fornecedor, produto e forma de pagamento; CRUD, busca e filtros.
Documentos	Máscaras e validação dos dígitos verificadores de CPF e CNPJ.
Compra	Fornecedor, itens, valores e entrada automática no estoque.
Venda	Cliente, itens, formas de pagamento, cálculo e baixa automática do estoque.
Históricos	Listagem de compras e vendas, detalhes por botão ou duplo clique.
Relatórios	Vendas por período e ranking dos dez produtos mais vendidos.
Permissões	Somente administradores podem excluir registros.
Aparência	Tema claro/escuro, botões padronizados e tabelas filtráveis.

Regras de estoque

Ao registrar uma compra, a quantidade comprada é somada ao estoque do produto.

Ao registrar uma venda, a quantidade vendida é subtraída do estoque.

A movimentação acontece na mesma transação do cabeçalho e dos itens.

Integridade

Se uma etapa da compra ou venda falhar, o DAO executa rollback. Assim, o estoque não fica alterado sem que o movimento tenha sido gravado por completo.

3. Como configurar e usar

3.1 Configuração da conexão

As credenciais são lidas de `src/main/resources/config.properties`. O arquivo deve conter URL JDBC, usuário, senha e SSL. A senha real não deve ser publicada no GitHub.

```
db.url=jdbc:postgresql://HOST:6543/postgres
db.user=postgres.PROJECT_REF
db.password=SUA_SENHA
db.ssl=true
```

3.2 Compilação e execução

```
mvn clean package
java -jar target/marketplace-erp.jar

# Alternativa durante o desenvolvimento
mvn exec:java
```

3.3 Fluxo recomendado de uso

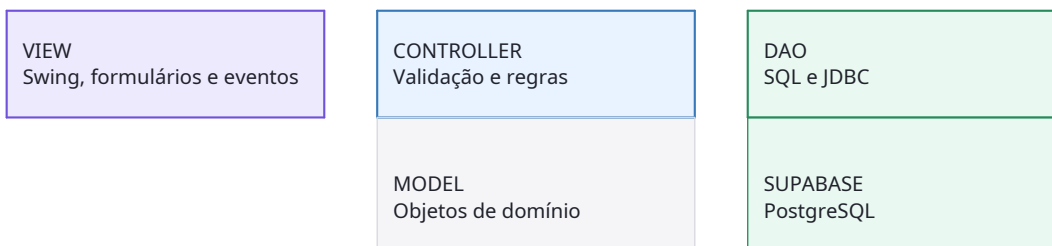
- 1 Iniciar o sistema e autenticar-se.
- 2 Cadastrar usuários e marcar “Perfil administrador” quando necessário.
- 3 Cadastrar clientes, fornecedores, produtos e formas de pagamento.
- 4 Registrar compras para alimentar o estoque.
- 5 Registrar vendas, incluindo produtos e formas de pagamento.
- 6 Consultar os históricos e abrir os detalhes do movimento.
- 7 Gerar relatórios de vendas por período e produtos mais vendidos.
- 8 Alternar o tema no menu Exibir e trocar de usuário quando necessário.

Permissão

Um usuário comum pode consultar e cadastrar, mas recebe uma mensagem de permissão negada ao tentar excluir.

4. Arquitetura MVC

A aplicação separa responsabilidades em pacotes. Essa divisão reduz acoplamento e facilita localizar cada tipo de regra.



Pacote	Responsabilidade
model	Entidades/POJOs: Pessoa, Cliente, Produto, Venda, Compra etc.
view	Login, menu, cadastros, movimentos, históricos, detalhes e relatórios.
controller	Validação, cálculos e coordenação entre View e DAO.
dao	Comandos SQL, transações, ResultSet e persistência.
util	Conexão, sessão, permissões, tema, máscaras e validações.

Regra de separação

A View não deve executar SQL. O Controller não precisa conhecer DriverManager. O DAO não deve controlar componentes Swing.

5. Padrão DAO — foco principal

DAO significa Data Access Object. O padrão cria uma camada exclusiva para persistência. Cada DAO sabe como inserir, atualizar, excluir, buscar e listar uma entidade, além de executar consultas específicas.

5.1 Por que usar DAO?

- Evita SQL espalhado pelas telas e controllers.
- Reduz o acoplamento entre interface gráfica e banco de dados.
- Centraliza tratamento de SQLException e fechamento de recursos.
- Permite reutilizar consultas em diferentes telas.
- Facilita alterar tabelas e comandos sem reescrever a interface.
- Torna explícitas as fronteiras das transações.

5.2 Contrato GenericDAO

```
public interface GenericDAO<T> {  
    int inserir(T entidade);  
    void atualizar(T entidade);  
    void excluir(int codigo);  
    T buscarPorCodigo(int codigo);  
    List<T> listar();  
}
```

O tipo genérico T representa a entidade manipulada. Assim, ClienteDAO implements GenericDAO<Cliente> e ProdutoDAO implements GenericDAO<Produto> seguem o mesmo contrato CRUD.

Classe DAO	Responsabilidade principal
UsuarioDAO	CRUD de usuários e autenticação por login/senha.
PessoaDAO	Operações auxiliares usadas por ClienteDAO e FornecedorDAO.
ClienteDAO	CRUD de Cliente junto com a Pessoa associada.
FornecedorDAO	CRUD de Fornecedor junto com a Pessoa associada.
ProdutoDAO	CRUD de Produto e movimentação de estoque.
FormaPagamentoDAO	CRUD das formas de pagamento.
VendaDAO	Venda, itens, pagamentos, detalhes e baixa de estoque.
CompraDAO	Compra, itens, detalhes e entrada de estoque.
RelatorioDAO	Consultas agregadas de vendas e produtos.

5.3 Como um DAO funciona internamente

Abertura de conexão

O DAO solicita uma conexão à fábrica. Ele não conhece senha, URL ou detalhes do driver.

```
try (Connection conn = ConnectionFactory.getConnection();
     PreparedStatement ps = conn.prepareStatement(sql)) {
    // parâmetros e execução
}
```

PreparedStatement

Os valores são passados por parâmetros ?. Isso evita concatenar dados no SQL, melhora o tratamento de tipos e reduz o risco de SQL injection.

```
String sql = "SELECT * FROM usuario "
            + "WHERE usu_login=? AND usu_senha=? AND usu_ativo='S'";
ps.setString(1, login);
ps.setString(2, senha);
ResultSet rs = ps.executeQuery();
```

Mapeamento ResultSet Model

A linha retornada é convertida em um objeto. As camadas superiores recebem Usuario, Cliente ou Produto, e não um ResultSet.

```
Usuario u = new Usuario();
u.setCodigo(rs.getInt("usu_codigo"));
u.setNome(rs.getString("usu_nome"));
u.setLogin(rs.getString("usu_login"));
u.setAtivo(rs.getString("usu_ativo"));
u.setAdmin(rs.getString("usu_admin"));
return u;
```

Fechamento automático

O bloco try-with-resources fecha Connection, PreparedStatement e ResultSet mesmo quando ocorre uma exceção. Operações transacionais também usam um bloco finally para restaurar o auto-commit e fechar a conexão.

6. Transações e consistência

Transação é um conjunto de comandos que deve terminar por inteiro ou ser totalmente desfeito. No projeto, isso é essencial porque algumas operações atualizam várias tabelas.

6.1 Cliente + Pessoa

- 1 ClienteDAO abre uma conexão e executa `setAutoCommit(false)`.
- 2 PessoaDAO.inserir usa a mesma conexão e devolve `pes_codigo`.
- 3 O DAO insere o Cliente com a chave da Pessoa.
- 4 Se tudo funcionar, executa `commit()`.
- 5 Se qualquer SQL falhar, executa `rollback()`.

```
conn = ConnectionFactory.getConnection();
conn.setAutoCommit(false);

int pesCodigo = pessoaDAO.inserir(conn, cliente.getPessoa());
// INSERT em cliente usando pesCodigo

conn.commit();
// no catch: rollback(conn)
```

Benefício

Não fica uma Pessoa órfã caso a inserção de Cliente falhe. O mesmo princípio é usado em Fornecedor.

6.2 Venda completa

- 1 Inserir o cabeçalho na tabela venda e obter o código com RETURNING.
- 2 Inserir cada item em `venda_produto`.
- 3 Subtrair a quantidade do estoque pelo ProdutoDAO.
- 4 Inserir as formas de pagamento em `venda_pagto`.
- 5 Executar `commit` somente depois de todas as etapas.

```
for (VendaProduto item : venda.getItems()) {
    // INSERT em venda_produto
    produtoDAO.movimentarEstoque(
        conn,
        item.getProduto().getCodigo(),
        item.getQtde().negate()
    );
}
conn.commit();
```

6.3 Compra e outras consultas DAO

Compra

A compra segue o mesmo modelo da venda, porém a quantidade é somada ao estoque. Cabeçalho, itens e estoque são confirmados juntos.

Detalhamento de movimentos

VendaDAO.buscarPorCodigo carrega cabeçalho, cliente, usuário, itens e formas de pagamento.

CompraDAO.buscarPorCodigo carrega cabeçalho, fornecedor, usuário e itens.

Os diálogos de detalhes apenas exibem os objetos retornados pelos controllers.

Relatórios no DAO

O RelatorioDAO contém consultas agregadas. A soma e o agrupamento são feitos pelo PostgreSQL, que é mais apropriado para processar conjuntos de registros.

```
SELECT vda_data,  
       COUNT(*) AS qtd,  
       COALESCE(SUM(vda_total), 0) AS total  
FROM venda  
WHERE vda_data BETWEEN ? AND ?  
GROUP BY vda_data  
ORDER BY vda_data;
```

```
SELECT pr.pro_nome,  
       SUM(vp.vep_qtde) AS qtd,  
       SUM(vp.vep_total) AS total  
FROM venda_produto vp  
JOIN produto pr ON pr.pro_codigo = vp.pro_codigo  
GROUP BY pr.pro_codigo, pr.pro_nome  
ORDER BY qtd DESC, total DESC  
LIMIT ?;
```

DAOException

Erros JDBC são convertidos em DAOException com contexto, por exemplo “Erro ao registrar venda”. Dessa forma, a View mostra uma mensagem útil sem depender diretamente de SQLException.

7. ConnectionFactory, JDBC e Supabase

ConnectionFactory é uma classe utilitária responsável por carregar configurações, registrar o driver PostgreSQL e abrir conexões. A configuração é lida uma única vez do classpath.

```
Properties connProps = new Properties();
connProps.setProperty("user", user);
connProps.setProperty("password", password);
connProps.setProperty("sslmode", "require");
connProps.setProperty("prepareThreshold", "0");
return DriverManager.getConnection(url, connProps);
```

Por que prepareThreshold=0?

O projeto utiliza o pooler transacional do Supabase, baseado em PgBouncer. A configuração desativa prepared statements do lado do servidor no driver JDBC, evitando conflitos de nomes de statements reutilizados pelo pooler. O código continua usando PreparedStatement normalmente para parâmetros e segurança.

Importante

Nenhum DAO contém credenciais. Todos chamam ConnectionFactory.getConnection(). A senha deve ficar apenas na configuração local e nunca no repositório.

Fluxo completo de uma ação

- 1 O usuário clica em Salvar na View.
- 2 A View monta uma entidade Model e chama o Controller.
- 3 O Controller valida os dados e chama o DAO.
- 4 O DAO abre conexão, prepara SQL e executa a operação.
- 5 O DAO mapeia o resultado e devolve objetos ou códigos.
- 6 O Controller devolve o resultado à View, que atualiza a tabela ou mostra mensagem.

8. Banco de dados e relacionamentos

Tabela	Papel	Relacionamentos principais
pessoa	Dados pessoais e de contato.	Base para cliente e fornecedor.
cliente	Cliente e limite de crédito.	FK para pessoa; referenciado por venda.
fornecedor	Fornecedor e contato.	FK para pessoa; referenciado por compra.
usuario	Login, situação e perfil admin.	Referenciado por compra e venda.
produto	Preço, custo, estoque e unidade.	Referenciado pelos itens.
formapagto	Meios de pagamento.	Referenciado por venda_pagto.
venda	Cabeçalho da venda.	Cliente, usuário, itens e pagamentos.
venda_produto	Itens vendidos.	FK para venda e produto.
venda_pagto	Pagamentos da venda.	FK para venda e forma de pagamento.
compra	Cabeçalho da compra.	Fornecedor, usuário e itens.
compra_produto	Itens comprados.	FK para compra e produto.

As tabelas de itens e pagamentos possuem vínculo com o movimento principal. O schema usa ON DELETE CASCADE nesses relacionamentos, permitindo remover automaticamente os filhos quando a compra ou venda é excluída.

Observação sobre exclusão

A permissão de administrador é verificada pela aplicação antes de chamar o DAO. As chaves estrangeiras continuam sendo a proteção estrutural da integridade no banco.

9. Segurança, permissões e recursos auxiliares

9.1 Sessão e autorização

Sessao mantém o usuário autenticado. Permissao.isAdmin() verifica o campo usu_admin. Antes de excluir, as telas chamam Permissao.podeExcluir(...); usuários comuns recebem um aviso e a operação não chega ao DAO.

```
public static boolean isAdmin() {  
    Usuario u = Sessao.getUsuarioLogado();  
    return u != null && u.isAdmin();  
}
```

9.2 Máscaras e validações

Mascaras: CPF/CNPJ, telefone, data e moeda brasileira.

ValidadorDocumento: algoritmo dos dígitos verificadores de CPF e CNPJ.

TableRowSorter e RowFilter: busca dinâmica nas tabelas de cadastro.

9.3 Tema

Tema instala FlatLaf, centraliza cores e fontes, estiliza botões/tabelas e alterna entre modo claro e escuro em tempo de execução.

9.4 Recomendações para produção

Armazenar senhas com hash seguro, nunca em texto puro.

Aplicar autorização também no servidor ou banco, não apenas na interface.

Usar variáveis de ambiente ou cofre de segredos para credenciais.

Adicionar pool de conexões dedicado em cenários com muitos usuários.

Registrar logs e criar testes automatizados para regras críticas.

10. Roteiro de apresentação e conclusão

Roteiro sugerido

- 1 Apresentar o objetivo do ERP e as tecnologias utilizadas.
- 2 Mostrar os pacotes Model, View, Controller, DAO e util.
- 3 Abrir GenericDAO e explicar o contrato CRUD.
- 4 Abrir ClienteDAO e demonstrar a transação Pessoa + Cliente.
- 5 Abrir VendaDAO e explicar cabeçalho, itens, pagamentos e estoque no mesmo commit.
- 6 Mostrar ConnectionFactory e a configuração externa do Supabase.
- 7 Executar login e cadastrar um produto, cliente e fornecedor.
- 8 Registrar uma compra e confirmar a entrada no estoque.
- 9 Registrar uma venda e confirmar a baixa no estoque.
- 10 Abrir históricos, detalhes e relatórios.
- 11 Demonstrar que um usuário comum não consegue excluir registros.

Resumo da importância do DAO

Mensagem principal para a apresentação

O DAO é a camada que protege o restante da aplicação dos detalhes do banco. Ele concentra SQL, parâmetros, conexões, ResultSet, transações e tratamento de erros. Graças a isso, controllers e telas trabalham com objetos do domínio e permanecem organizados.

O trabalho demonstra uma aplicação completa de orientação a objetos integrada a banco relacional. O uso de MVC organiza o fluxo da interface até os dados, enquanto o DAO reduz o acoplamento e deixa claras as responsabilidades de persistência. As transações em Cliente, Fornecedor, Compra e Venda preservam a consistência mesmo quando uma operação envolve várias tabelas.

Arquivos principais para estudar: GenericDAO.java, ClienteDAO.java, VendaDAO.java, CompraDAO.java, RelatorioDAO.java, ConnectionFactory.java e os Controllers correspondentes.